Wine Quality Prediction

Regression with physicochemical measurements | Aivancity PGE 2

Project report · 30 October 2026

Task	Predict the numerical wine quality score with regression.
Dataset	UCI Wine Quality, red and white Vinho Verde samples.
Selected model	KNeighborsRegressor, 21 neighbours, distance weights.
Final test RMSE	0.685 quality-score points.

Summary

This project estimates the quality score of red and white Vinho Verde wines from laboratory measurements. The task remains regression throughout. The code combines two UCI files, adds a red/white category, removes exact duplicate rows, and compares a mean baseline with five regression algorithms. A scikit-learn Pipeline performs imputation, scaling and encoding inside cross-validation. Model choice uses training data only. The selected pipeline achieved lower error than the DummyRegressor on the final held-out test split, while its R^2 shows that much of the score variation remains unexplained.

The implementation is a small Python package with a command-line interface, tests, an executed notebook and saved model. The project is intended as coursework, so the report explains the decisions and their limits instead of treating the model as a production rating system.

Source: Cortez et al. (2009), UCI Machine Learning Repository, dataset 186, DOI 10.24432/C56S3T. The dataset is licensed CC BY 4.0.

1. Data and exploration

The local raw files contain 1,599 red and 4,898 white samples, for 6,497 rows in total. Each file has 11 physicochemical predictors and the `quality` target. The combined table adds `wine_type`, giving 12 input columns. All input columns are numeric except the red/white category. The target is an integer score. The source files have zero missing cells.

Exact duplicates were counted after adding wine type. There were 1,177 repeated rows, leaving 5,320 rows. This avoids identical records crossing the train/test boundary. It may also remove genuinely separate samples that happen to share the same recorded values, so the decision is stated as a limitation.

Quality is concentrated at 5 and 6. Scores range from 3 to 9, and the dataset has only five examples at score 9. As a result, the model has much less evidence for the rarest scores. Red and white observations are not evenly represented: white wines make up about three quarters of the combined data.

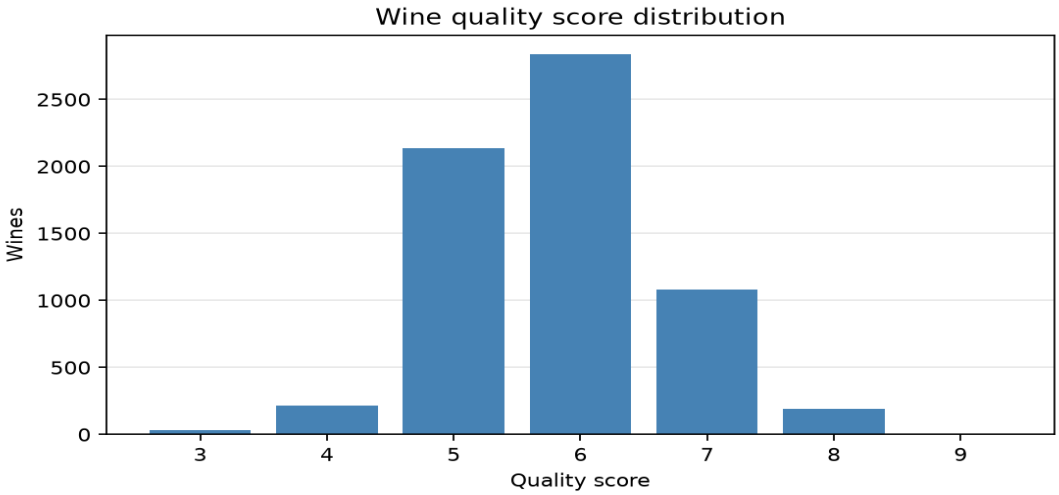


Figure 1. Target score counts after combining the two files and before duplicate removal.

No missing values required correction in this dataset. The numeric imputer remains in the reusable pipeline so future input data with missing predictors can be handled. Exploratory plots and correlation summaries are included in the notebook and `reports/figures/`.

2. Method

The cleaned rows were split once into 4,256 training observations and 1,064 test observations, using a 20% test size and random seed 42. The test subset remained unused during cross-validation and hyperparameter search. This separation makes the final score a less biased estimate of generalization than a score measured on the data used for model choice.

A ColumnTransformer treats the two input types. Numeric predictors use median imputation followed by StandardScaler. `wine_type` uses most-frequent imputation and one-hot encoding. These transformations sit inside each model Pipeline, so every cross-validation training fold learns its own preprocessing values. This prevents information from the validation fold leaking into training.

The comparison includes a mean DummyRegressor, LinearRegression, Ridge, Lasso, LinearSVR and KNeighborsRegressor. Five-fold shuffled KFold cross-validation with seed 42 evaluates the training split. GridSearchCV tunes the regularized linear models, LinearSVR and k-NN. The lowest mean cross-validation RMSE determines the selected model. MAE and R^2 are reported alongside RMSE.

RMSE squares errors before averaging, so large mistakes contribute more. MAE is the average absolute error and is easier to explain in quality points. R^2 compares squared error with a mean prediction reference. An R^2 below zero would indicate worse performance than that reference on the evaluated data.

3. Results

KNeighborsRegressor achieved the lowest cross-validation RMSE at 0.715, with 21 neighbours and distance weighting. Its mean cross-validation MAE was 0.553 and R^2 was 0.343. The selected model was then fitted on all training rows and evaluated once on the untouched test set.

Model	CV RMSE	CV MAE	CV R^2
KNeighborsRegressor	0.715	0.553	0.343
Ridge	0.735	0.567	0.306
LinearRegression	0.735	0.567	0.306
Lasso	0.735	0.567	0.305
LinearSVR	0.737	0.566	0.302
DummyRegressor	0.883	0.697	-0.002

Test model	RMSE	MAE	R^2
DummyRegressor	0.868	0.694	-0.001
KNeighborsRegressor	0.685	0.534	0.375

On the held-out test data, the model reduced RMSE by about 21% compared with the mean baseline. Its MAE of 0.534 means the average absolute error was just over half a quality point. R^2 of 0.375 indicates that considerable variation remains unexplained.

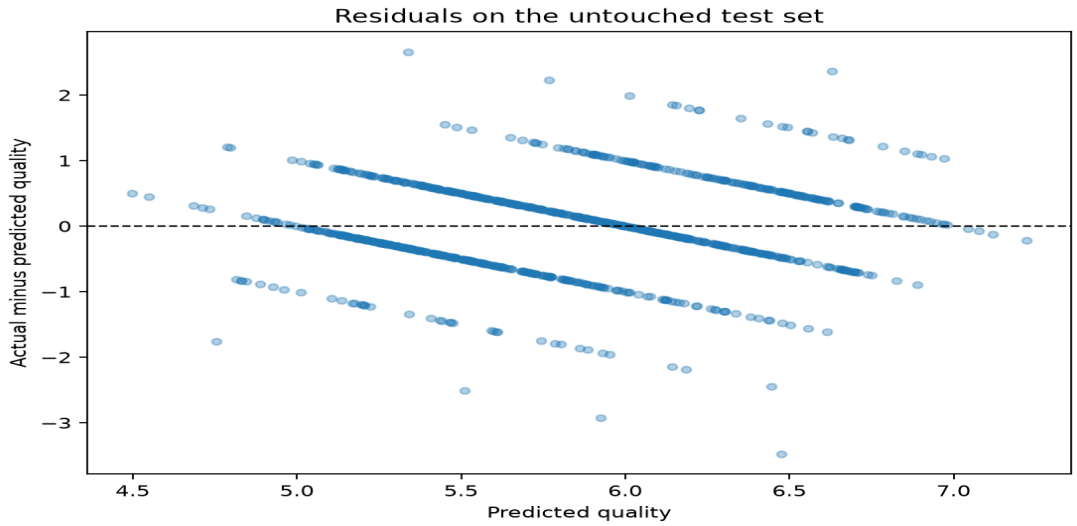


Figure 2. Residuals for the selected model on the final test split.

4. Interpretation, limitations and reproducibility

Permutation importance on the held-out rows ranked alcohol highest, followed by volatile acidity, free sulfur dioxide and sulphates. This method shuffles a feature and measures the decline in model score. It describes how this fitted model uses the available columns; it does not establish that any measurement causes a quality change. Correlated predictors can also share or obscure importance.

Feature	Mean RMSE increase after shuffling
alcohol	0.108
volatile_acidity	0.039
free_sulfur_dioxide	0.036
sulphates	0.033
total_sulfur_dioxide	0.033

The prediction can be fractional even though the original quality score is an integer. The data cover Portuguese Vinho Verde wines and omit grape variety, producer, price, tasting conditions and other context. The score is ordinal, so treating the gap from 3 to 4 like the gap from 7 to 8 is a simplification. Rare high scores also limit what the model can learn about that part of the scale.

The fitted preprocessing and k-NN estimator are saved together in a joblib file. The CLI exposes `train`, `evaluate` and `predict`. The executed notebook shows data inspection, plots, split, preprocessing, model comparison and the example prediction. The tests cover loading and validation, transforms, training, persistence, prediction, CLI behavior and EDA outputs.

Reproduction: activate the project virtual environment, run `python -m pip install -e ".[dev]"`, then run `wine-quality train`, `wine-quality evaluate` and `wine-quality predict --input examples/new_case.json`. Tests run with `pytest`; linting uses `ruff check src tests`. The exact split settings and full precision results are recorded in `config.yaml` and `reports/evaluation.json`.

Reference: Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). Wine Quality [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C56S3T.

AI assistance is disclosed in `AI_USAGE.md`. Students should review the code, reproduce the run and ensure that the statement matches their actual use before submission.